

Lecture 13:

Clustering Algorithms

ML/AI for Engineering Applications
ENV 6932

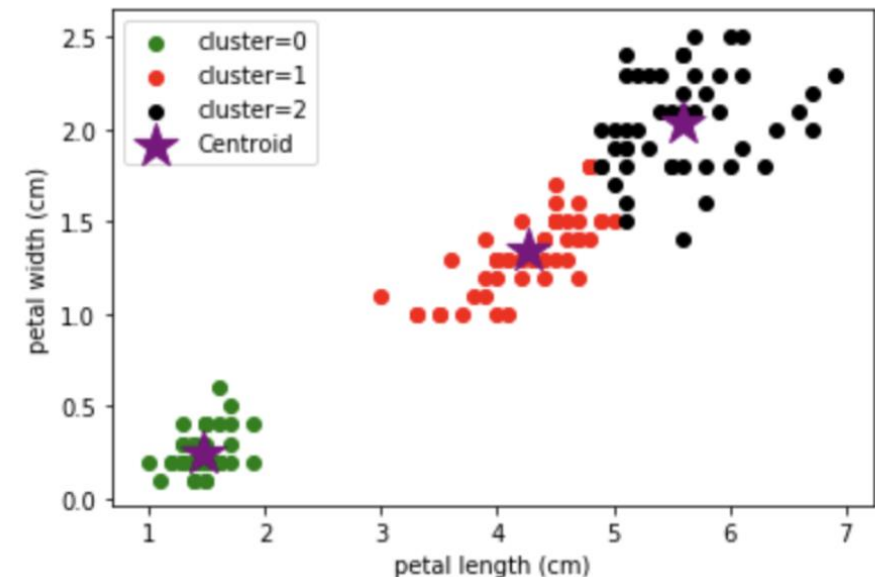
Sara Behdad
Fall 2025

Outline

- **Clustering Algorithms**
 - K-Means
 - Mean-shift
 - Density-Based Spatial Clustering of Applications with Noise (DBSCAN)
 - Agglomerative Hierarchical Clustering

Clustering Algorithms

- Clustering algorithms or Cluster analysis are unsupervised machine learning algorithms.
- They **automatically discover natural grouping in data**.
- Different from supervised learning, clustering only interprets the input data and find natural groups in feature space.



Clustering Algorithms

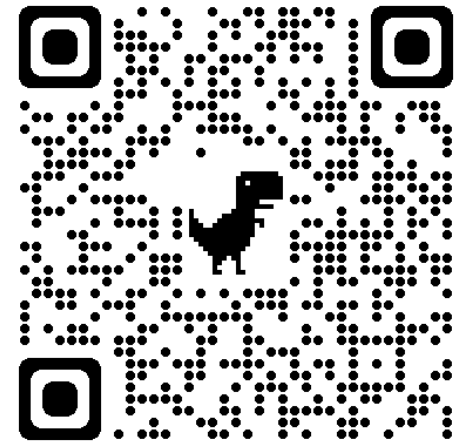
- K-Means Clustering
- Mean-Shift Clustering
- Density-Based Spatial Clustering of Applications with Noise (DBSCAN)
- Expectation–Maximization (EM) Clustering using Gaussian Mixture Models (GMM)
- Agglomerative Hierarchical Clustering

K-Means

- K-Means discovers underlying patterns in the dataset and groups similar data points together.
- It partitions n objects into k clusters in which each object belongs to the cluster with the nearest mean.
- It creates **sphere-shaped** clusters.
- K-Means is suitable for **medium and large** datasets.

K-Means

- K : The number of centroids we need in the dataset.
- Centroid: the imaginary or real location demonstrating the center of a cluster.



QR code for K-means example

K-Means

- The best k leading to the greatest separation (distance) is not known as a priori and must be computed from the data.
- The objective function:

The diagram shows the objective function J for K-Means clustering. The formula is $J = \sum_{j=1}^k \sum_{i=1}^n \|x_i^{(j)} - c_j\|^2$. Annotations include: 'number of clusters' pointing to k , 'number of cases' pointing to n , 'case i ' pointing to $x_i^{(j)}$, 'centroid for cluster j ' pointing to c_j , and 'Distance function' pointing to the norm $\|x_i^{(j)} - c_j\|^2$. The entire expression is labeled 'objective function' with an arrow pointing to J .

$$\text{objective function} \leftarrow J = \sum_{j=1}^k \sum_{i=1}^n \|x_i^{(j)} - c_j\|^2$$

number of clusters number of cases case i centroid for cluster j

Distance function

K-Means Algorithm Steps

- **Step 1:** *choose a value for the number of groups/clusters/classes, K*
- **Step 2:** randomly initialize K center points.
- **Step 3:** assign each point to a closest cluster by computing the distance between that point and each group center.
- **Step 4:** after all points are assigned to clusters, calculate the mean of each cluster and select the mean of each cluster as the new center point for that cluster (centroid), and go to Step 3.
- **Step 5:** Repeat Steps 3 and 4 for a predefined number of iterations or until the group centers don't change much between iterations.

Example: K-Means

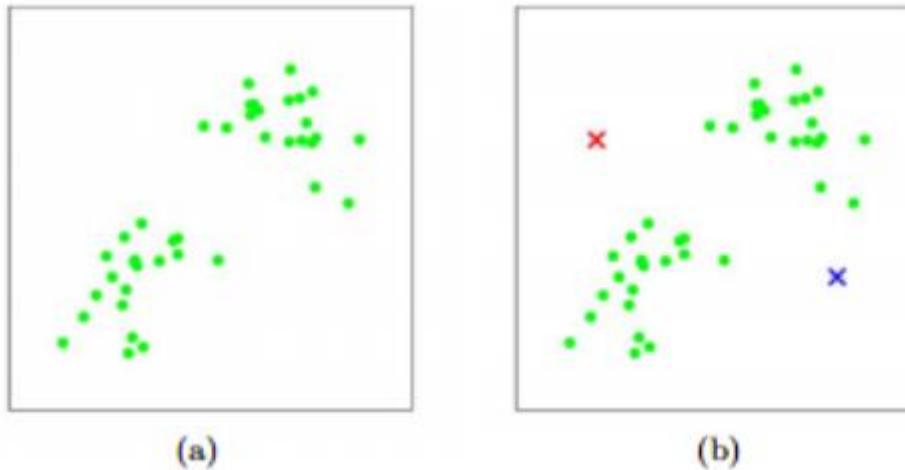
- Step 1: Select K, let's consider $K=2$



(a)

Example: K-Means

- Step 2: randomly initialize K center points.

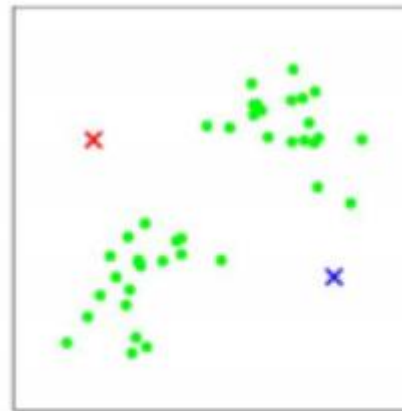


Example: K-Means

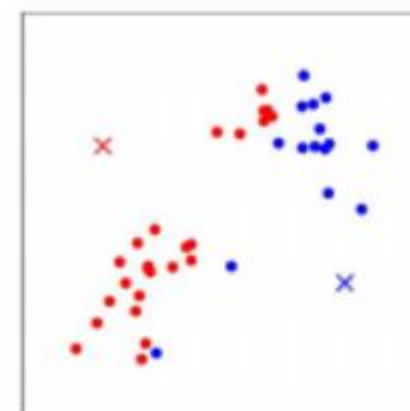
- Step 3: assign each point to a closest cluster by computing the distance between that point and each group center.



(a)



(b)



(c)

Example: K-Means

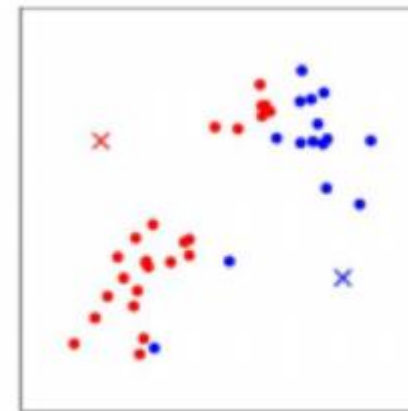
- Step 4: calculate the mean of each cluster and select the mean as the new centroid



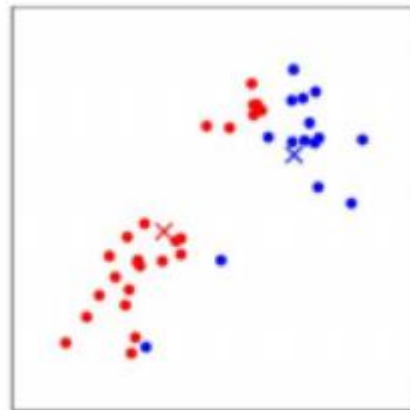
(a)



(b)



(c)



(d)

Example: K-Means

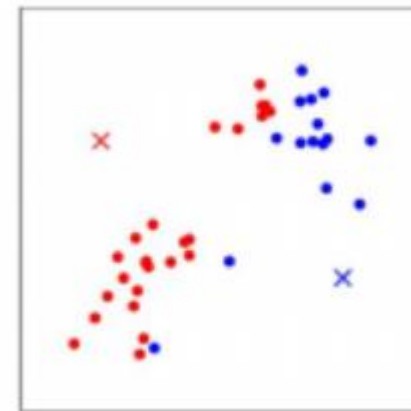
- Step 5: Repeat Steps 3 and 4 and reassign data points to clusters.



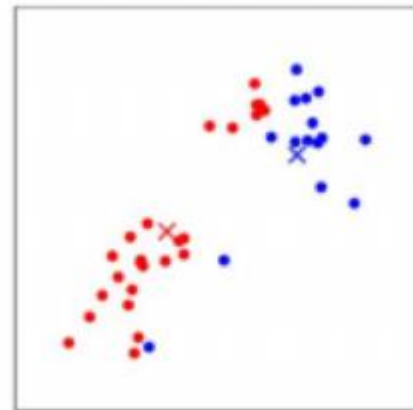
(a)



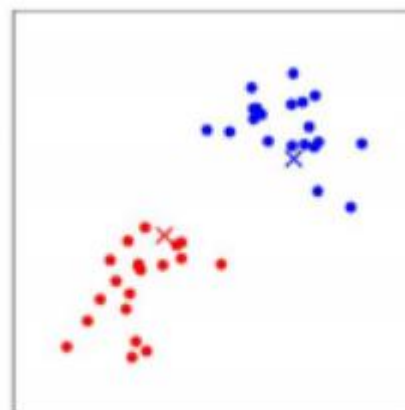
(b)



(c)



(d)



(e)

Example: K-Means

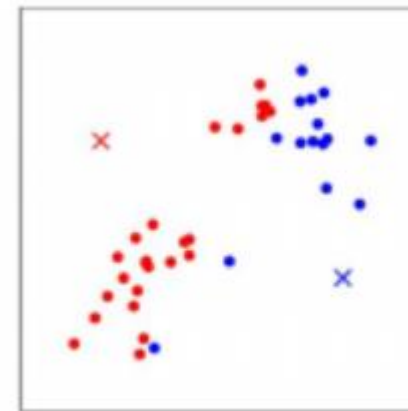
- Stopping point: stop when the group centers don't change much between iterations.



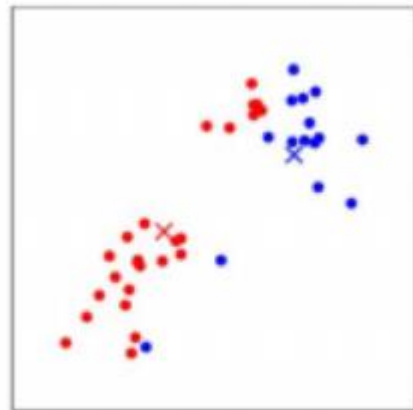
(a)



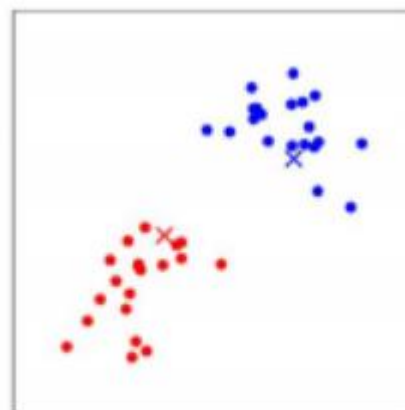
(b)



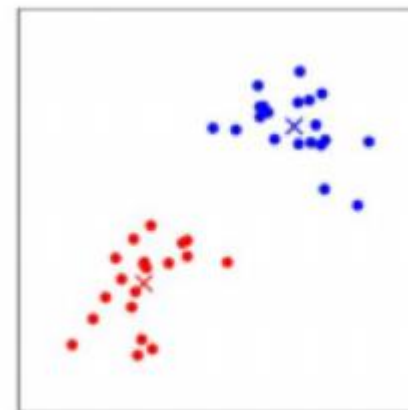
(c)



(d)



(e)



(f)

Mathematical Representation of K-Means

- **Objective:** to predict k centroids and a label for each datapoint.

1. Initialize **cluster centroids** $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^n$ randomly.

2. Repeat until convergence: {

For every i , set

$$c^{(i)} := \arg \min_j \|x^{(i)} - \mu_j\|^2.$$

For each j , set

$$\mu_j := \frac{\sum_{i=1}^m 1\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^m 1\{c^{(i)} = j\}}.$$

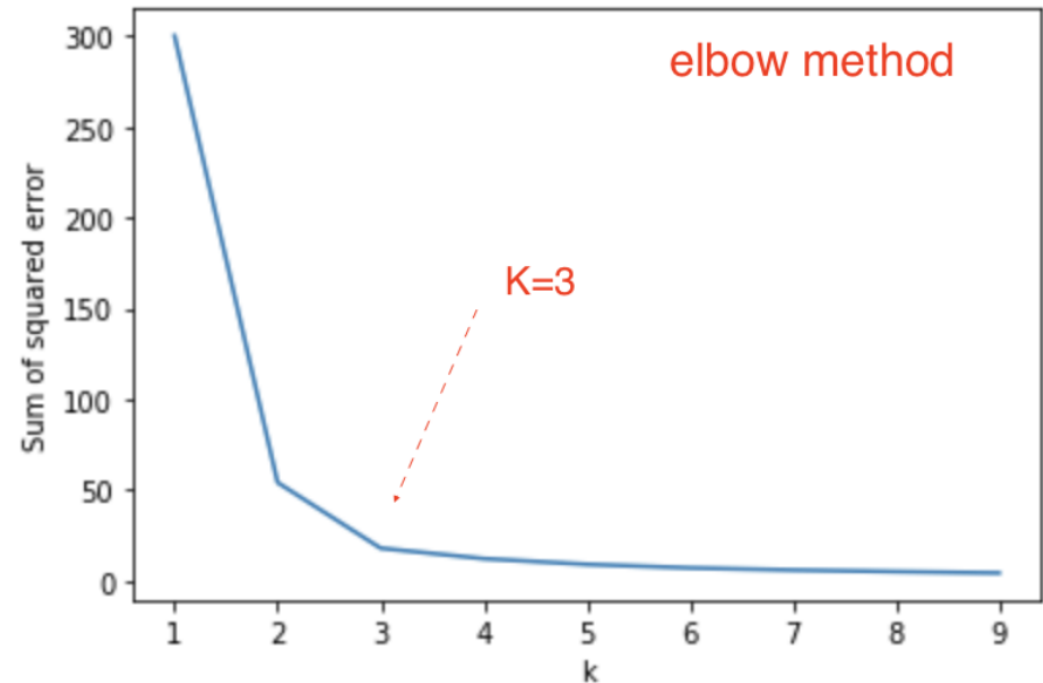
}

Evaluation Methods for K-Means

- Elbow method
- Silhouette analysis

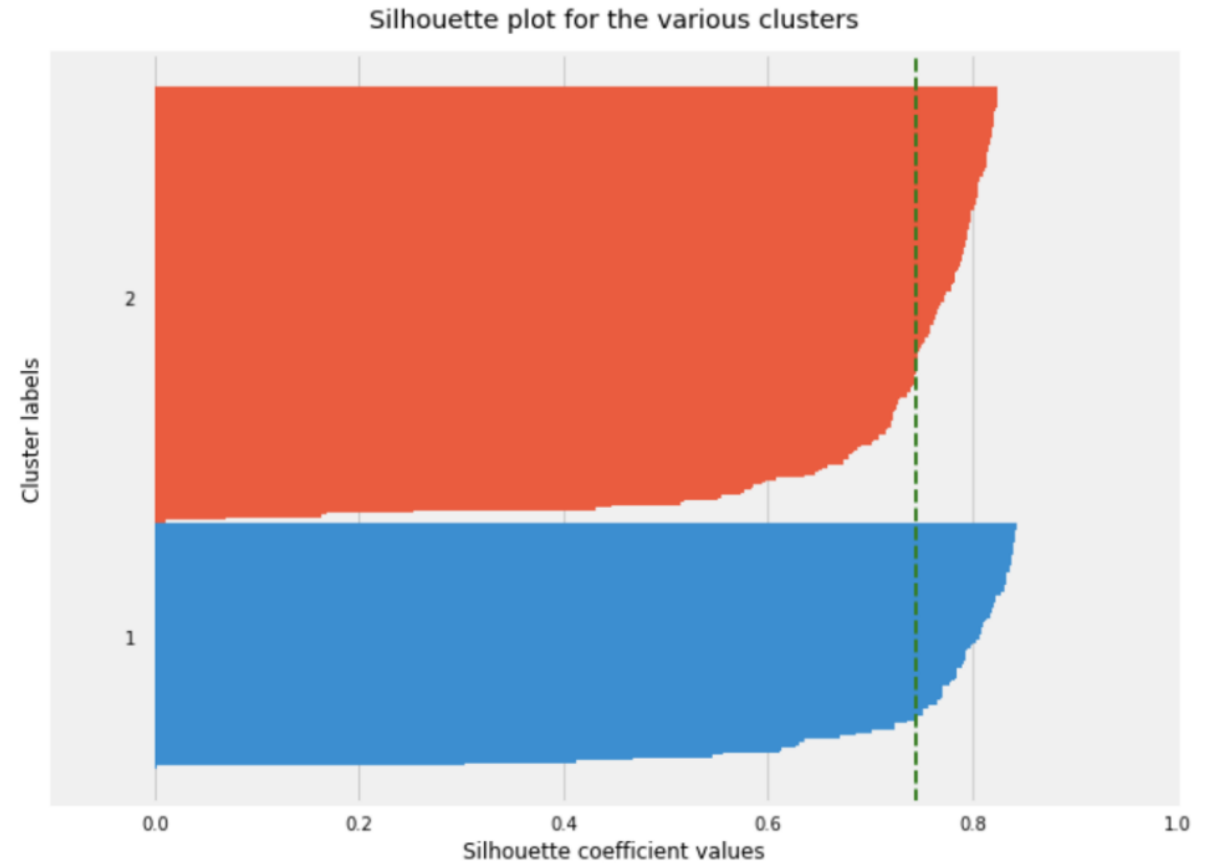
Evaluation Methods

- **Elbow method** uses the sum of squared distance (SSE) between data points and their assigned centroids.



Evaluation Methods

- Silhouette method



Evaluation Methods

- The silhouette coefficient is calculated as:

$$s(\mathbf{o}) = \frac{b(\mathbf{o}) - a(\mathbf{o})}{\max\{a(\mathbf{o}), b(\mathbf{o})\}}$$

- **$s(\mathbf{o})$** : the silhouette coefficient of the data point $\mathbf{o}$
- **$a(\mathbf{o})$** : the *average distance* between $\mathbf{o}$ and all the other data points in the cluster to which $\mathbf{o}$ belongs
- **$b(\mathbf{o})$** : the *minimum average distance* from $\mathbf{o}$ to all clusters to which $\mathbf{o}$ does not belong

Evaluation Methods

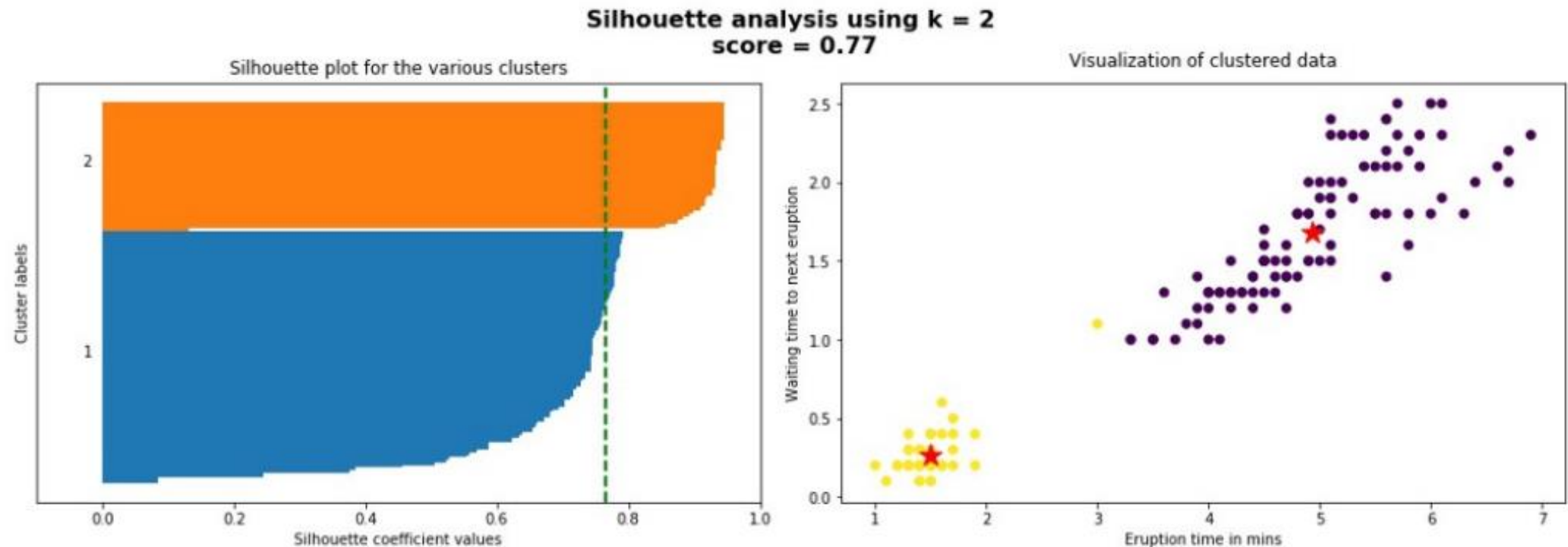
- Points about the silhouette coefficient :

$$s(\mathbf{o}) = \frac{b(\mathbf{o}) - a(\mathbf{o})}{\max\{a(\mathbf{o}), b(\mathbf{o})\}}$$

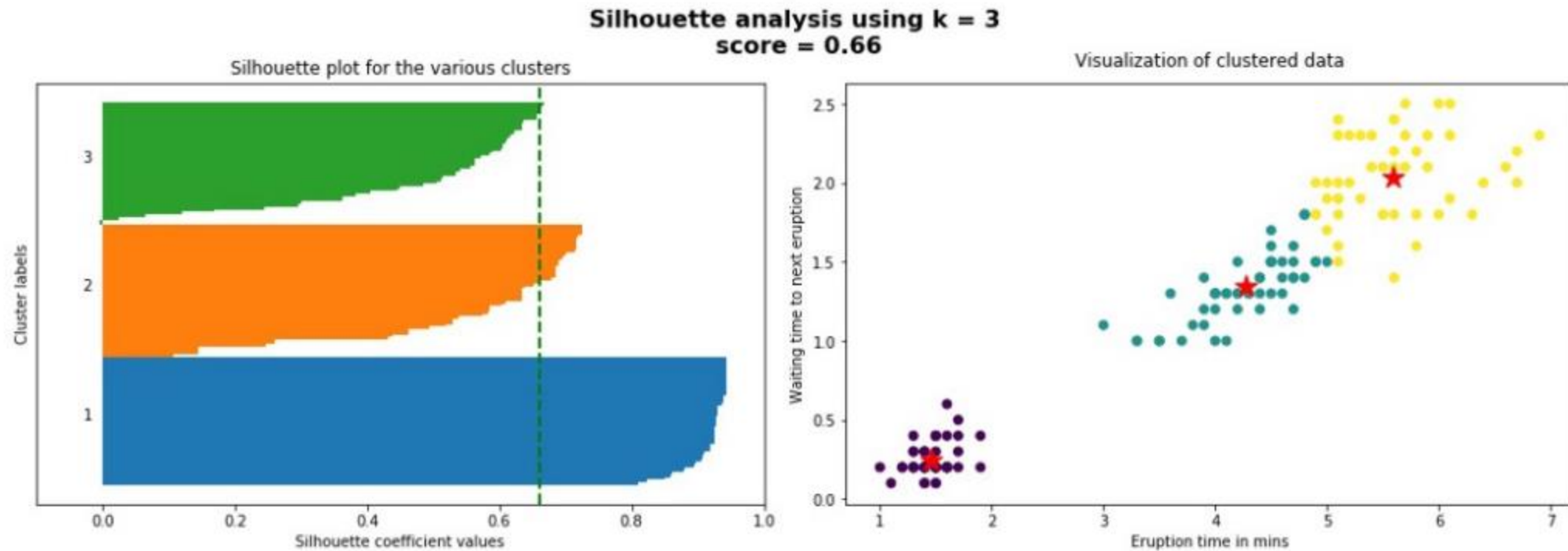
- The silhouette coefficient is between $[-1, 1]$.
- A score of **1** indicates the **best case** where the data point $\mathbf{o}$ is very compact within the cluster to which it belongs and far away from the other clusters.
- A score of **-1** shows the **worst** case.
- Values near **0** denote **overlapping** clusters.

Example of silhouette coefficient

- Most silhouette coefficients are above 0.8 when $k=2$:

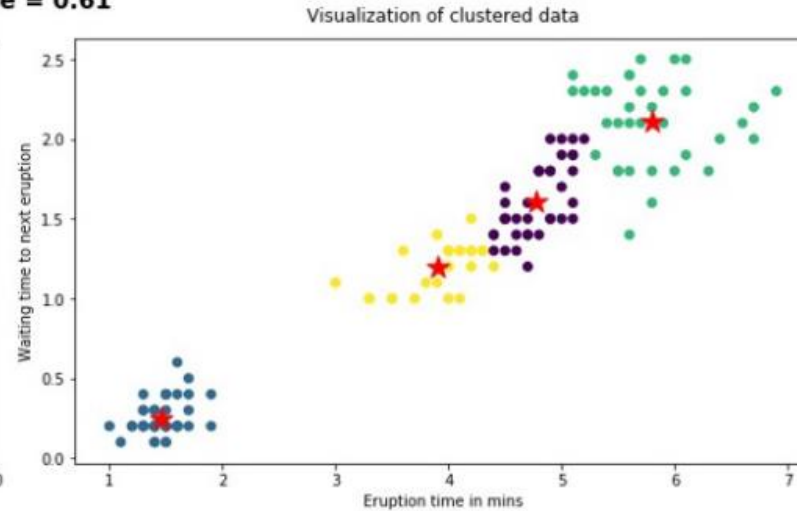
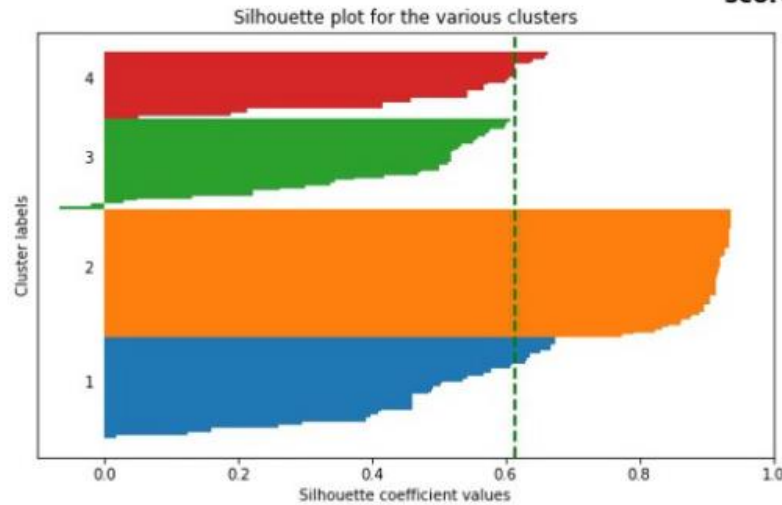


Example of silhouette coefficient

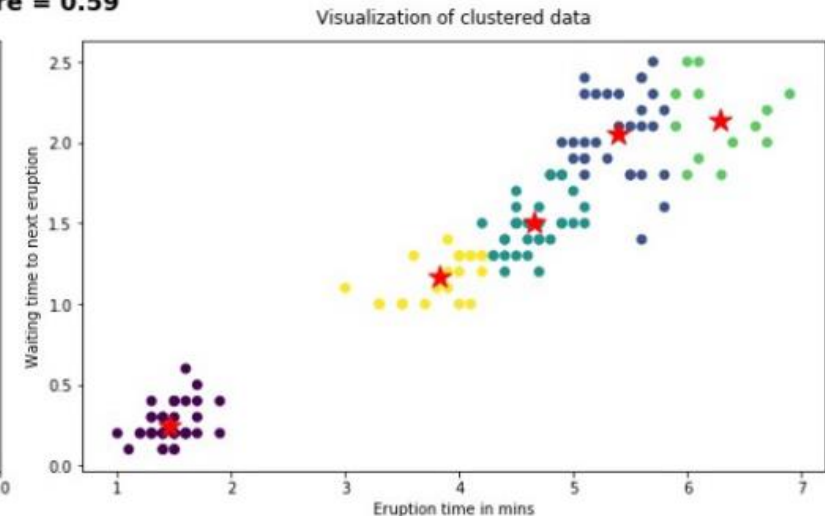
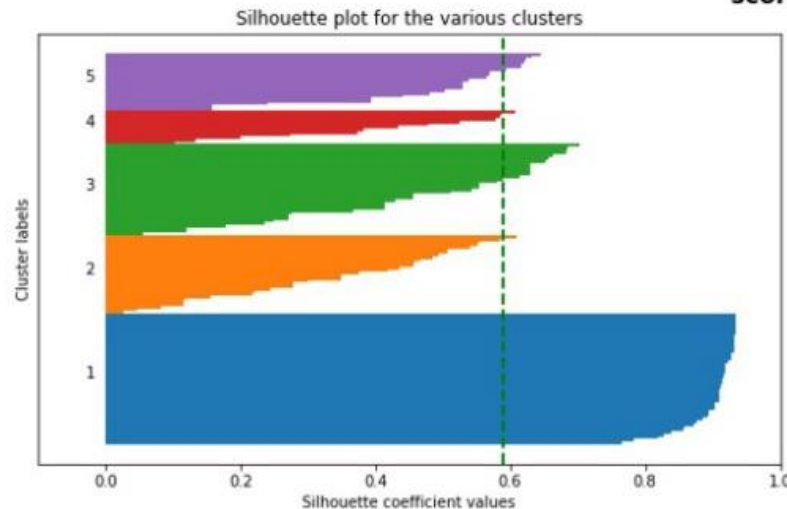


Example of silhouette coefficient

Silhouette analysis using k = 4
score = 0.61



Silhouette analysis using k = 5
score = 0.59

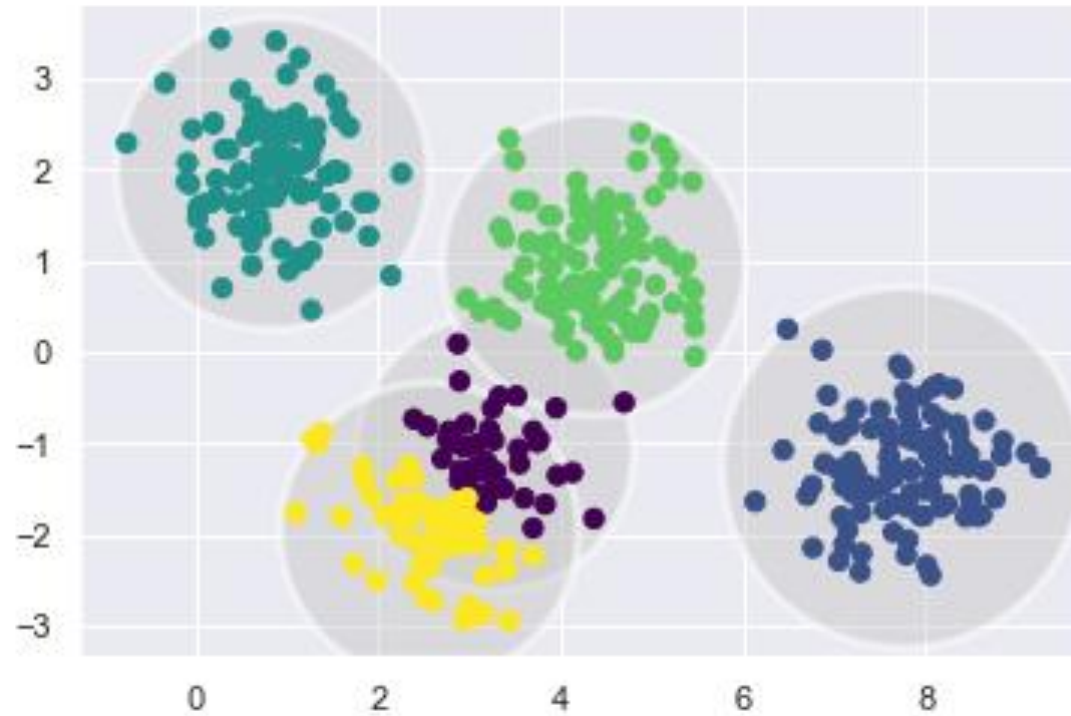


Disadvantages of K-Means

- The number of groups/classes (K) should be selected.
- It starts with a random choice of cluster centers, so it may result into different clustering outcomes on different runs of the algorithm (the results are not be repeatable, low consistency)
- Makes hard assignments of points to cluster.
- Sensitive to outliers.
- **K-Medians:** instead of recomputing the center points using the mean, the median is used (less sensitive to outliers but requires more computation, not efficient for larger datasets).

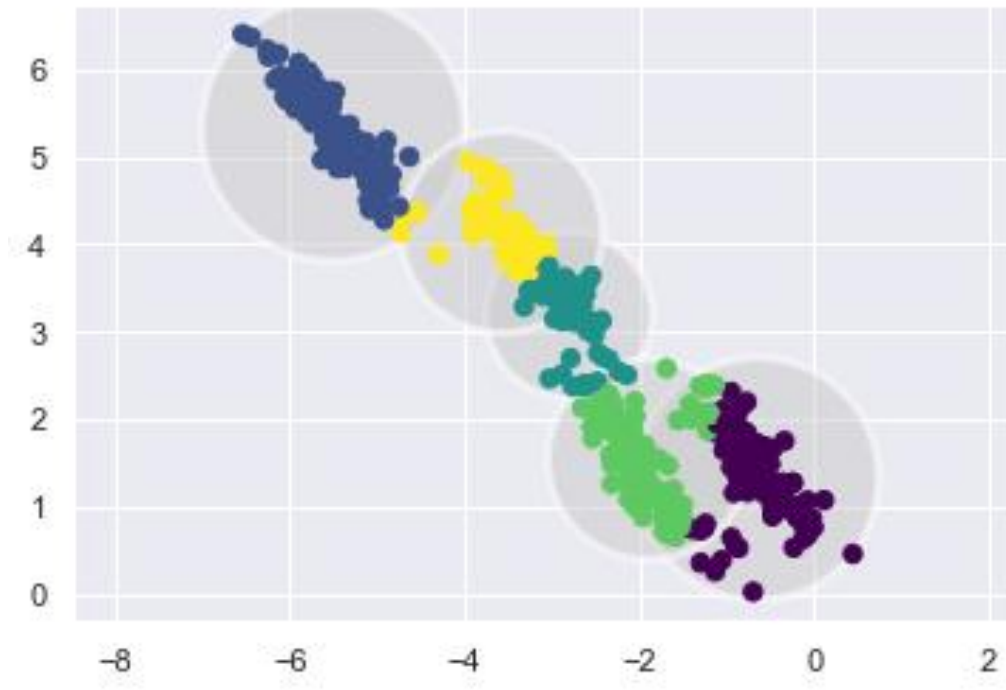
Weaknesses of K-means

- In K-means the cluster models must be circular



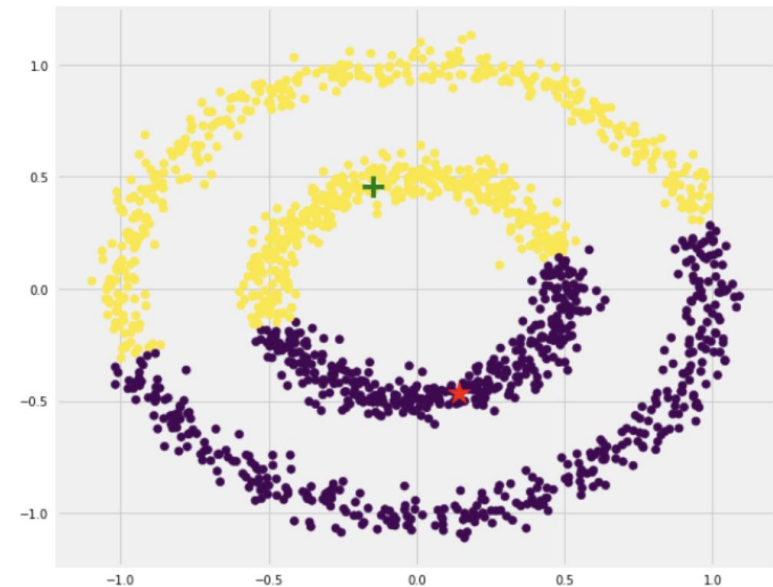
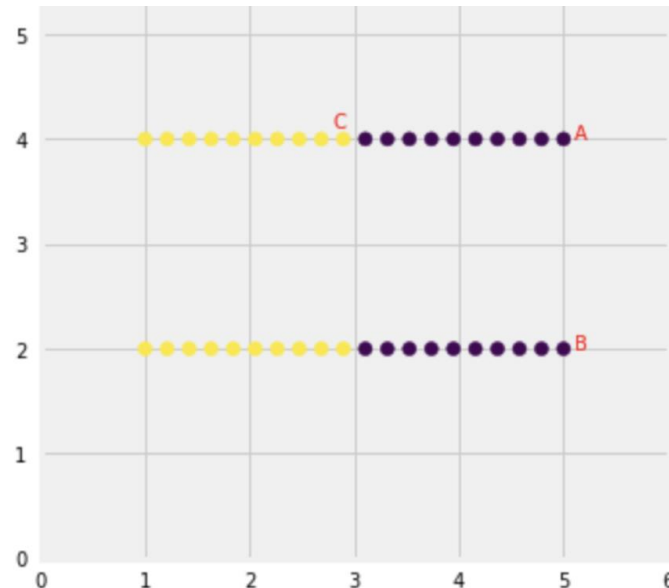
Let's transform the data

- Even visually we can see that a circular cluster is a poor fit.



More examples of k-means poor performance

- k-means can be viewed as a greedy algorithm since it tries to minimize the sum of the squared distances to the cluster centers.
- Poor job on non-convex shapes.

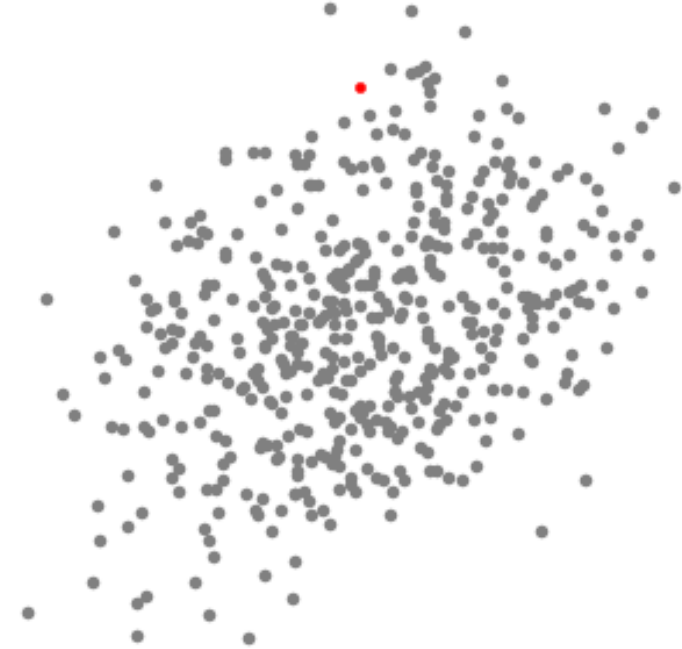


Clustering Algorithms

- K-Means Clustering
- **Mean-Shift Clustering**
- Density-Based Spatial Clustering of Applications with Noise (DBSCAN)
- Expectation-Maximization (EM) Clustering using Gaussian Mixture Models (GMM)
- Agglomerative Hierarchical Clustering

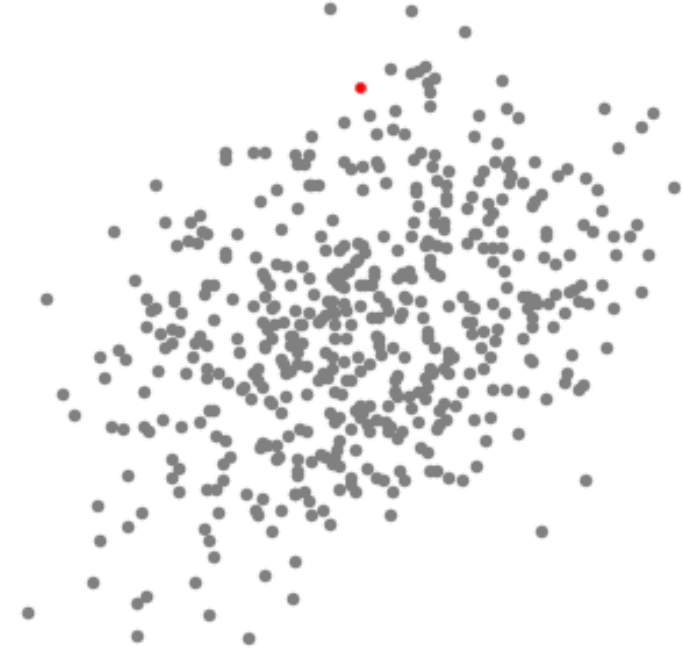
Mean-shift Clustering

- It is a sliding-window-based algorithm that attempts to find dense areas of data points.
- **Goal:** to locate the center points of each group/class (a centroid-based algorithm), which works by updating candidates for center points to be the mean of the points within the sliding-window.



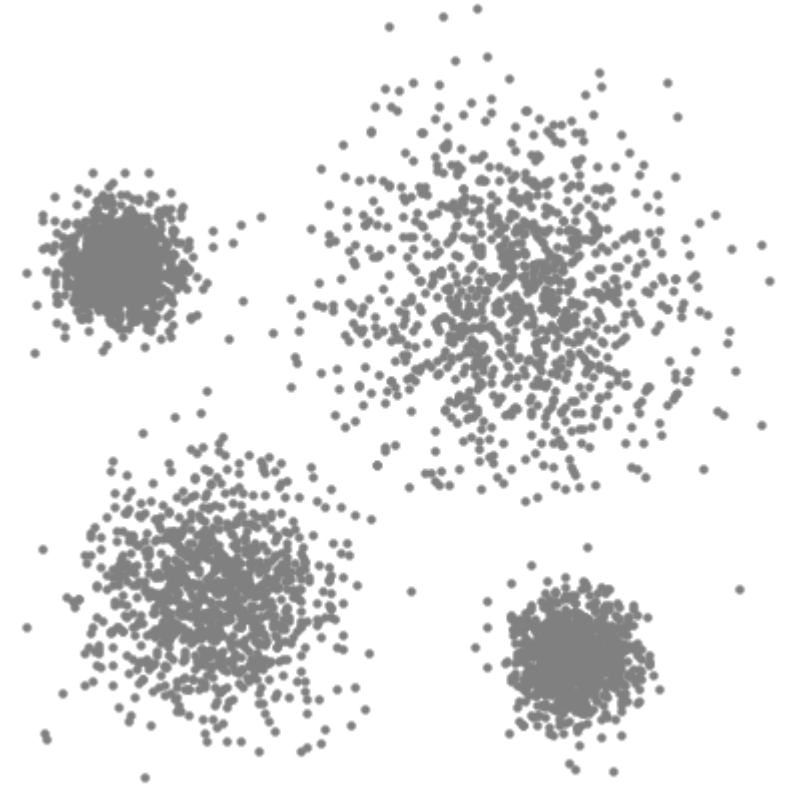
Mean-shift Clustering Steps

- **Step 1:** Randomly select a point C and begin with a circular sliding window centered at C and having radius r as the kernel.
- **Step 2:** At every iteration, the sliding window is shifted towards regions of higher density (this is a hill-climbing algorithm). This is done by shifting the center point to the mean of the points within the window.
- **Step 3:** Continue shifting the sliding window until there is no direction at which a shift can accommodate more points inside the kernel.



Mean-shift Clustering Steps

- Steps 1 to 3 is done with many sliding windows until all points lie within a window.
- When multiple sliding windows overlap, the window containing the most points is kept.
- The data points are then clustered according to the sliding window in which they reside.



Black dot: the centroid of a sliding window
Gray dot: a data point.



Mathematical Representation of Mean-shift Clustering

- **Recall:** Density Estimation discussion
- **Problem:** given discrete data sampled from a density function, how to locate the maxima (the modes) of a density function.
- Mean shift is an iterative process to find mode (**mode-seeking algorithm**).
- It starts with an initial estimate x . Let a kernel function be given.

$$K(x_i - x)$$

- This function determines the weight of nearby points for re-estimation of the mean. Typically, a Gaussian kernel on the distance to the current estimate is used:

$$K(x_i - x) = e^{-c\|x_i - x\|^2}$$

- The weighted mean of the density in the window determined by K is:

$$m(x) = \frac{\sum_{x_i \in N(x)} K(x_i - x)x_i}{\sum_{x_i \in N(x)} K(x_i - x)}$$

Mathematical Representation of Mean-shift Clustering

- The two most common kernels for mean shift:

- **Flat Kernel**

$$k(x) = \begin{cases} 1 & \text{if } x \leq \lambda \\ 0 & \text{if } x > \lambda \end{cases}$$

- **Gaussian Kernel:**

$$k(x) = e^{-\frac{x}{2\sigma^2}}$$

- At every iteration, the kernel is shifted to the centroid (the mean of the points within it).
- The method of calculating this mean depends on the choice of the kernel.
- If a Gaussian kernel is chosen, then every point will first be assigned a weight which will decay exponentially as the distance from the kernel's center increases.

Pros and Cons of Mean-shift Clustering

- **Advantages:**

- No assumption similar to K-means or Gaussian mixture.
- Can be used for data with nonconvex shape.
- It only requires one parameter, bandwidth, which automatically determines the number of clusters.
- No issue of local minima as we had in K-means.
- Outliers do not influence the final outcome.

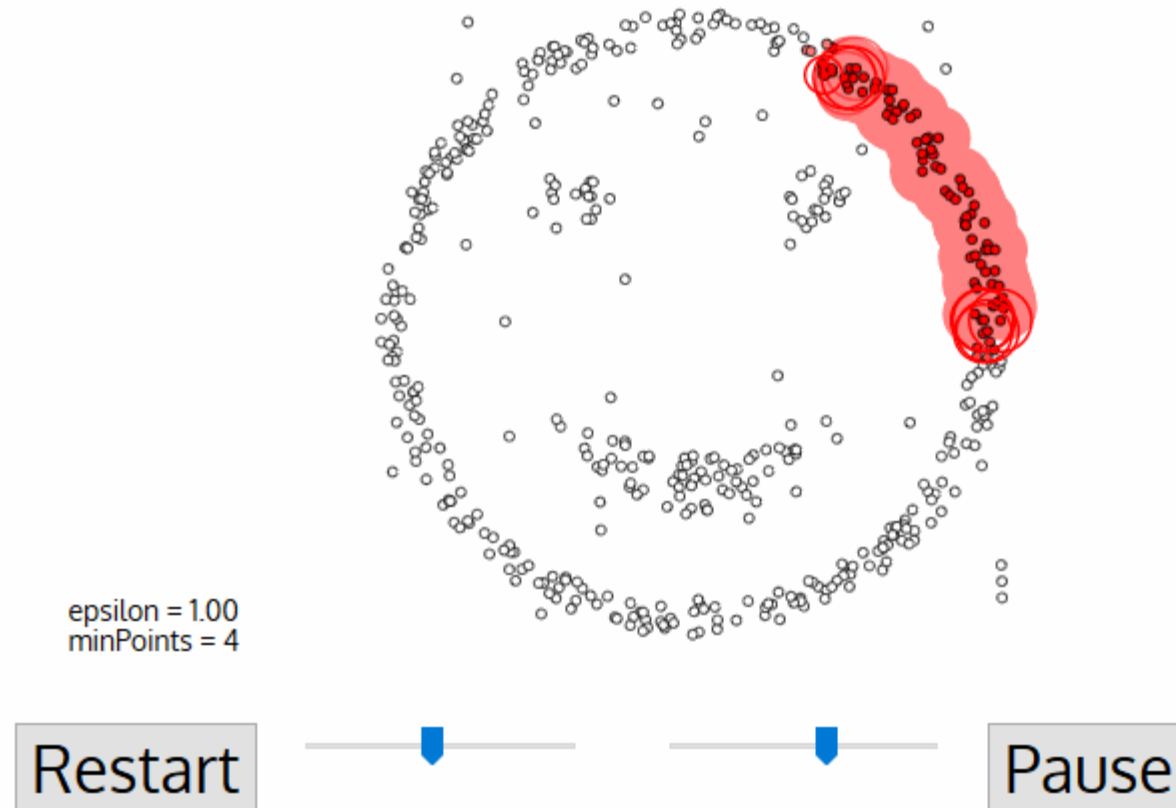
- **Disadvantages:**

- Computationally intensive
- No control on the number of clusters

Clustering Algorithms

- K-Means Clustering
- Mean-Shift Clustering
- **Density-Based Spatial Clustering of Applications with Noise (DBSCAN)**
- Agglomerative Hierarchical Clustering
- Expectation-Maximization (EM) Clustering using Gaussian Mixture Models (GMM)

DBSCAN

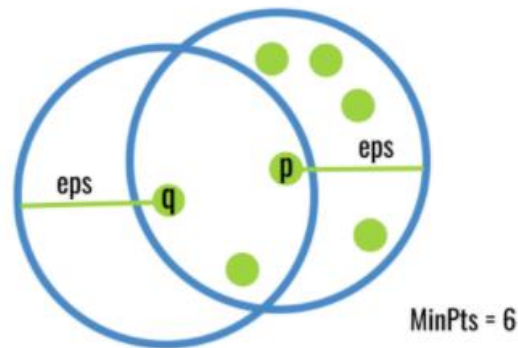


DBSCAN Steps

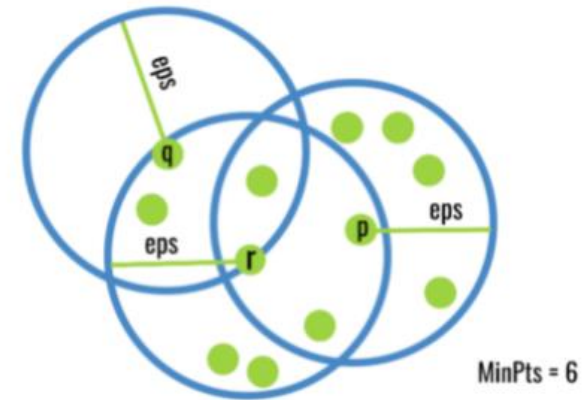
- Similar to Mean-Shift Clustering, DBSCAN is a density-based clustered algorithm.
- **Step 1:** Begin with an arbitrary starting data point (not been visited yet). Determine the neighborhood of this point (All points within the ϵ distance of this point).
- **Step 2:** If based on the minPoints criterion, there is a sufficient number of points within this neighborhood then the current data point becomes the first point in the new cluster. Otherwise, it is labeled as noise (later this noisy point might become the member of the cluster). In both cases that point is marked as “visited”.
- **Step 3:** For this first point, the points within its ϵ neighborhood also become part of the same cluster. This procedure of making all points in the ϵ neighborhood belong to the same cluster is then repeated for all of the new points that have been just added to the cluster group.
- Steps 2 and 3 is repeated until all points in the cluster are determined i.e all points have been visited and labeled.
- Once we're done with the current cluster, a new unvisited point is processed, to discovery of a further cluster or noise. This process repeats until all points are marked as visited.

Directly density reachable concept

- DBSCAN has a vital role in finding nonlinear shapes structure using the density.
- **Directly density reachable:** q is directly density reachable from p if q is within the ϵ -Neighborhood of p and p is a core point.



Directly density reachable



Density reachable

Pros and Cons of DBSCAN

- **Advantages:**

- No need for a pre-set number of clusters.
- It identifies outliers as noises, different from mean-shift which simply throws them into a cluster.
- It can recognize arbitrarily sized and arbitrarily shaped clusters quite well.

- **Disadvantages:**

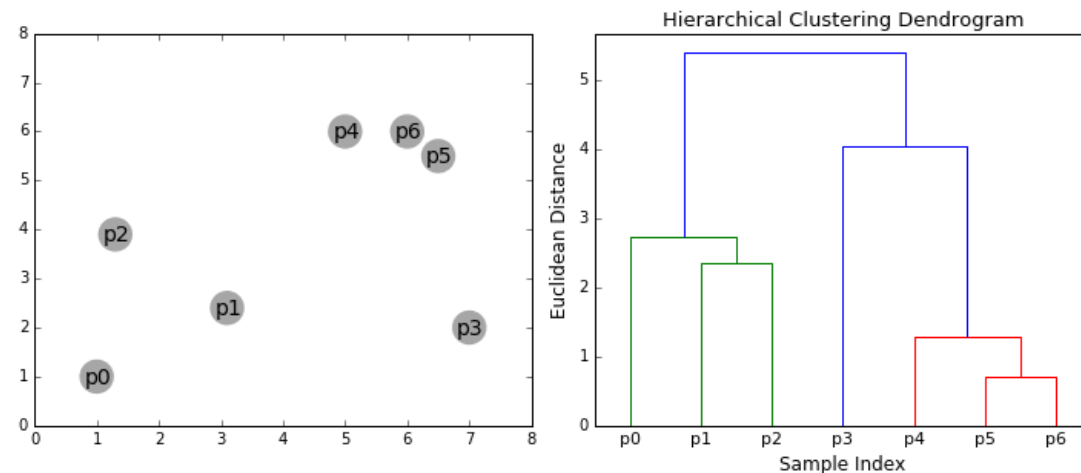
- It doesn't perform well when the clusters are of varying density.
- This is because the setting of ϵ and minPoints for identifying the neighborhood points will vary from cluster to cluster when the density varies.
- The distance ϵ is challenging to estimate in the case of very high-dimensional data.

Clustering Algorithms

- K-Means Clustering
- Mean-Shift Clustering
- Density-Based Spatial Clustering of Applications with Noise (DBSCAN)
- **Agglomerative Hierarchical Clustering**
- Expectation–Maximization (EM) Clustering using Gaussian Mixture Models (GMM)

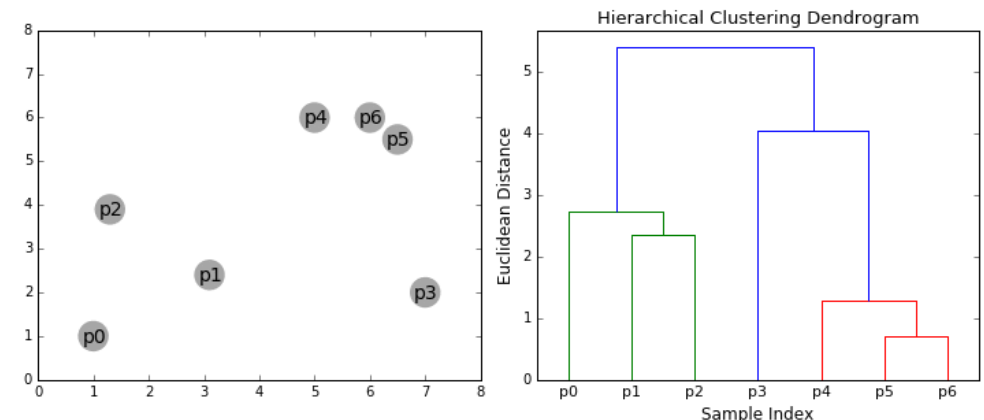
Agglomerative Hierarchical Clustering (HAC)

- Two categories of Hierarchical clustering algorithms:
 - Top-down (Divisive)
 - Bottom-up (Agglomerative)
- Bottom-up algorithms treat each data point as a single cluster and then successively merge pairs of clusters until all clusters have been merged into a single cluster containing all data points.
- The hierarchy of clusters is shown as a tree (or dendrogram).



Agglomerative Hierarchical Clustering Steps

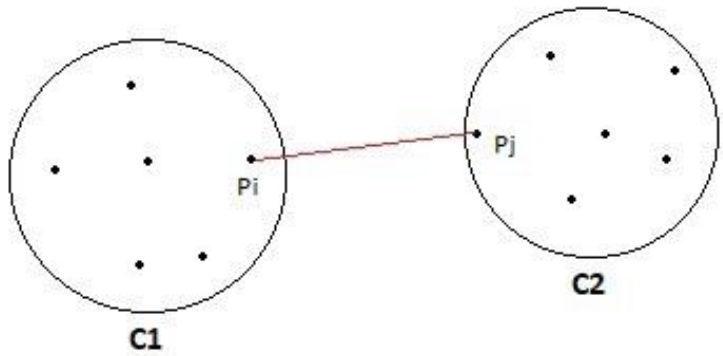
- **Step 1:** Begin by treating each data point as a single cluster (i.e., X data points, X clusters). Then, select a distance metric that measures the distance between two clusters (e.g., *average linkage*=distance between two clusters).
- **Step 2:** On each iteration, combine two clusters into one (combine clusters with the smallest average linkage based on your selected distance metric)
- Step 2 is repeated until we reach the root of the tree (i.e. only have one cluster containing all data points).



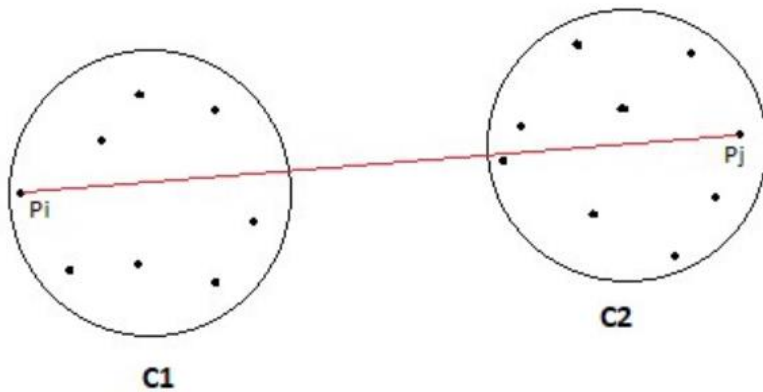
How to Calculate Linkage between Two Clusters?

- Certain approaches to calculate the linkage :
 - MIN
 - MAX
 - Group Average
 - Distance Between Centroids
 - Ward's Method

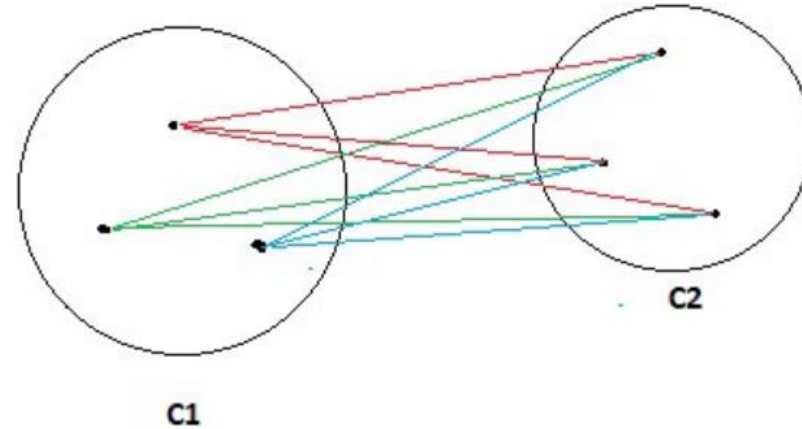
Methods to Calculate Linkage



MIN

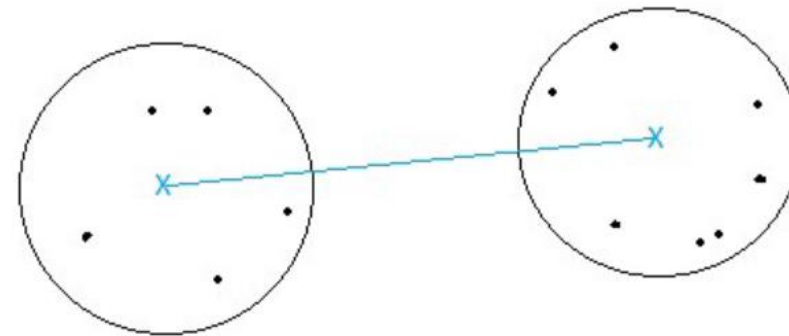


MAX



$$\text{sim}(C1, C2) = \sum \text{sim}(P_i, P_j) / |C1| * |C2|$$

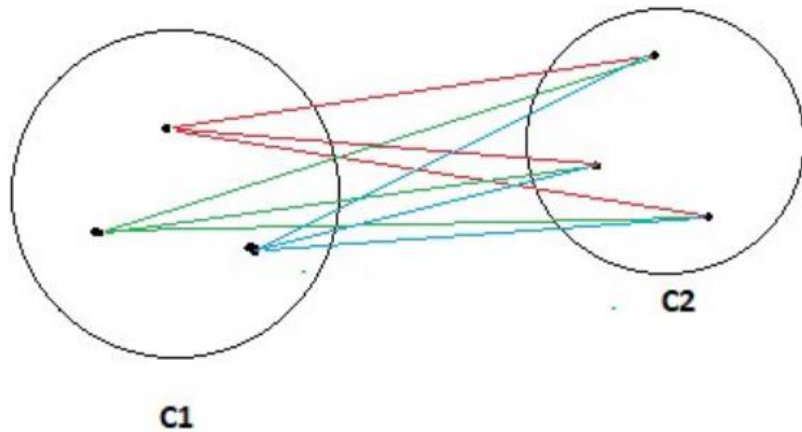
Group Average



Distance Between Centroids

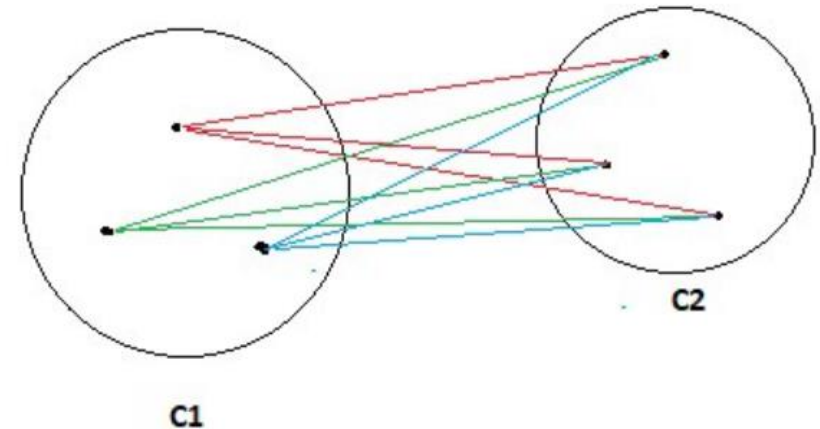
Ward's Method

- Ward's Method is exactly the same as Group Average except that it calculates the **sum of the square of the distances** P_i and P_j .



$$\text{sim}(C1, C2) = \sum \text{sim}(P_i, P_j) / |C1| * |C2|$$

Group Average



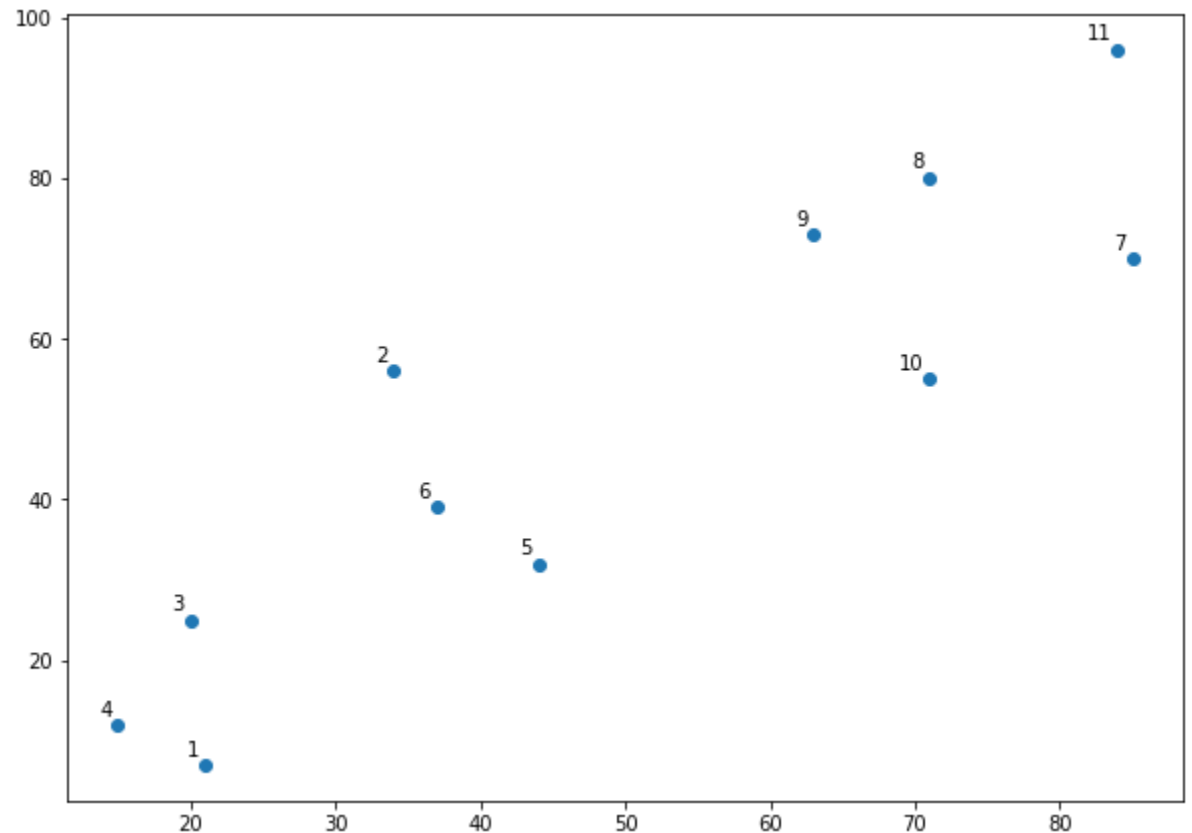
$$\text{sim}(C1, C2) = \sum (\text{dist}(P_i, P_j))^2 / |C1| * |C2|$$

Ward

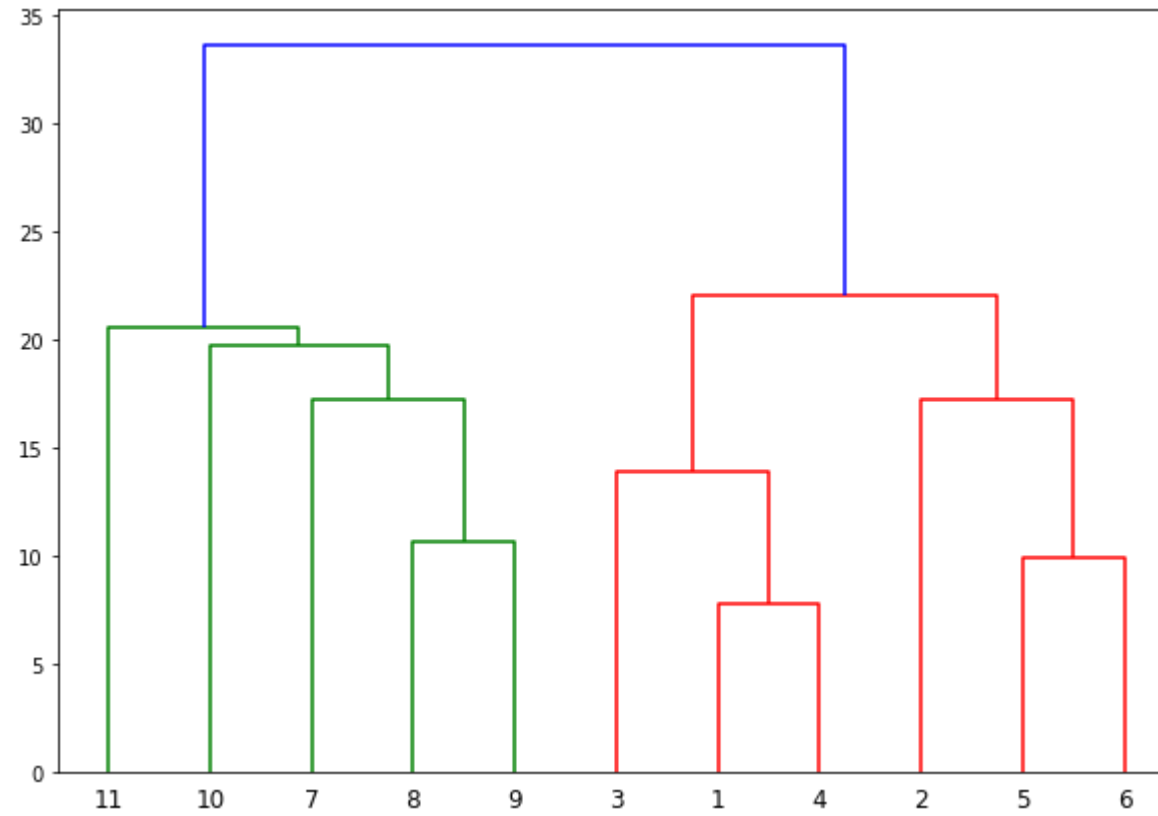
Example:

- Consider the following dataset: 11 data points

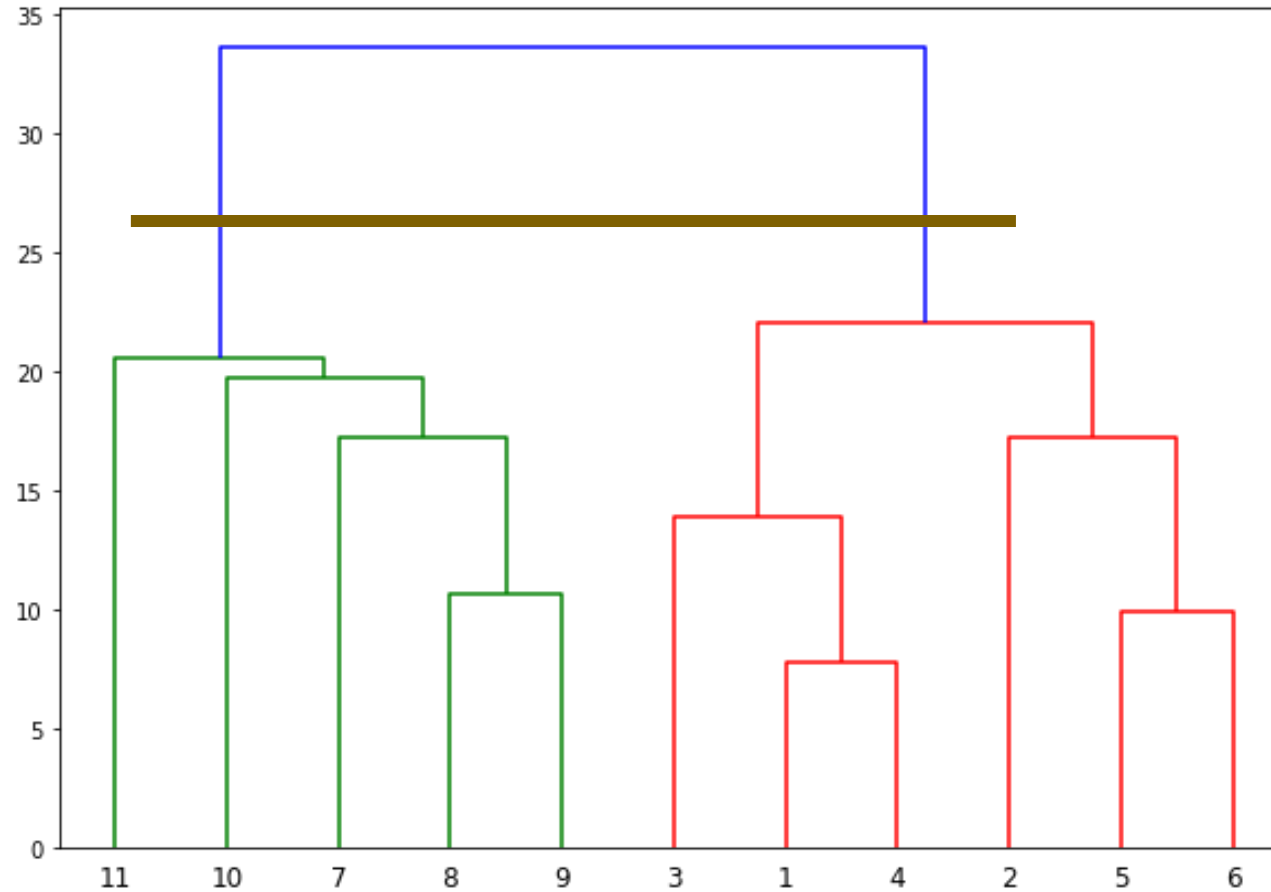
```
X = np.array([[21,7],  
[34,56],  
[20,25],  
[15,12],  
[44,32],  
[37,39],  
[85,70],  
[71,80],  
[63,73],  
[71,55],  
[84,96],])
```



Example

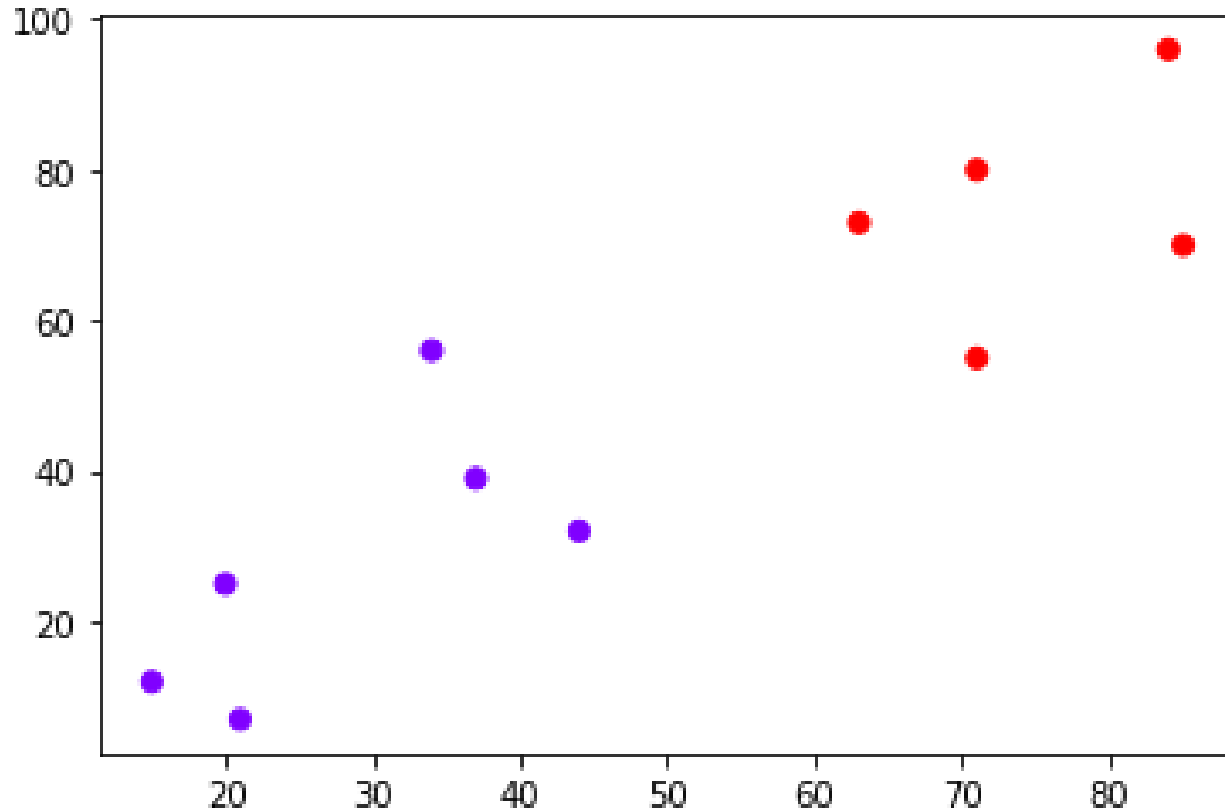


Number of clusters



[0 0 0 0 0 0 1 1 1 1 1]

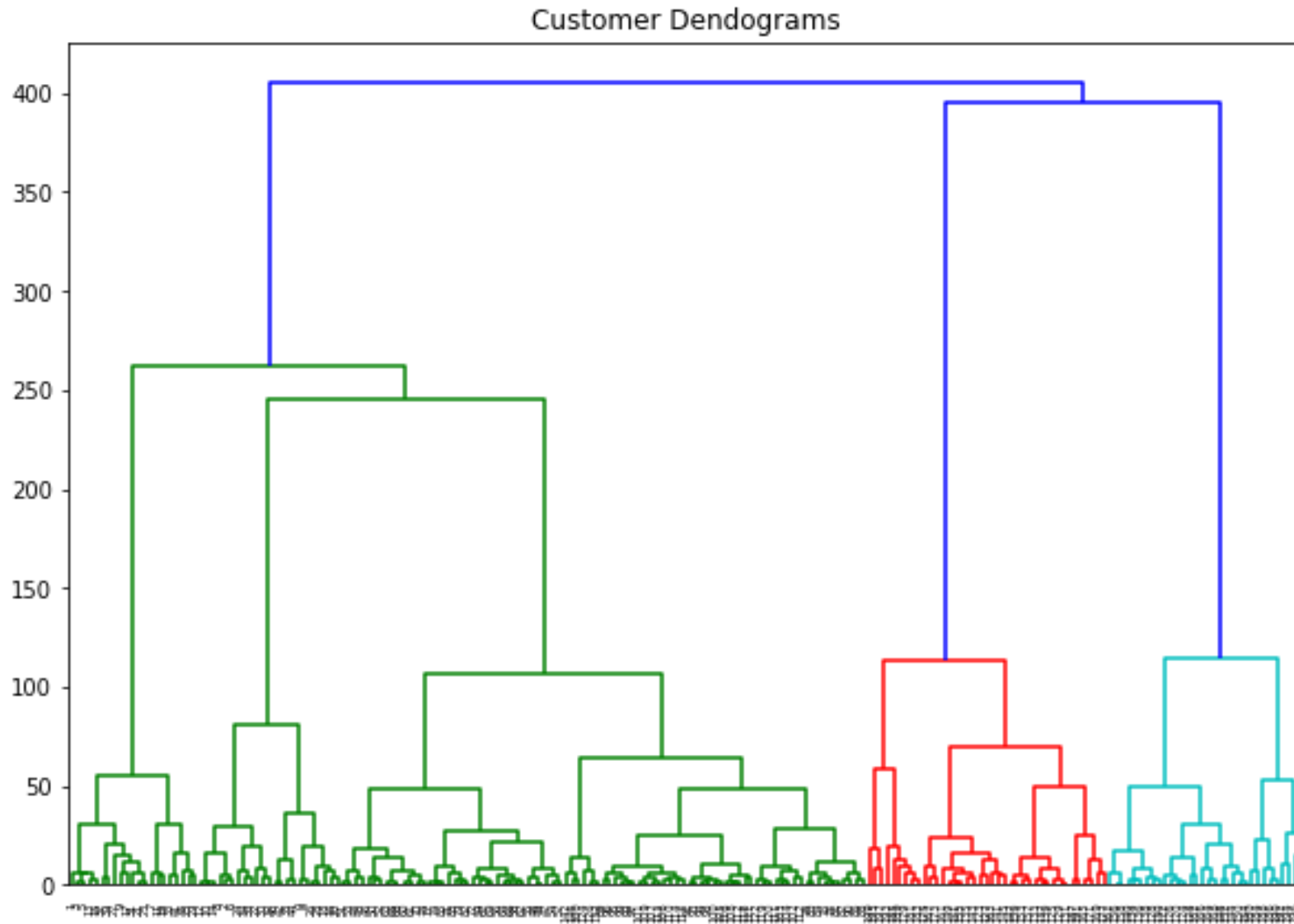
Assign Points to Clusters



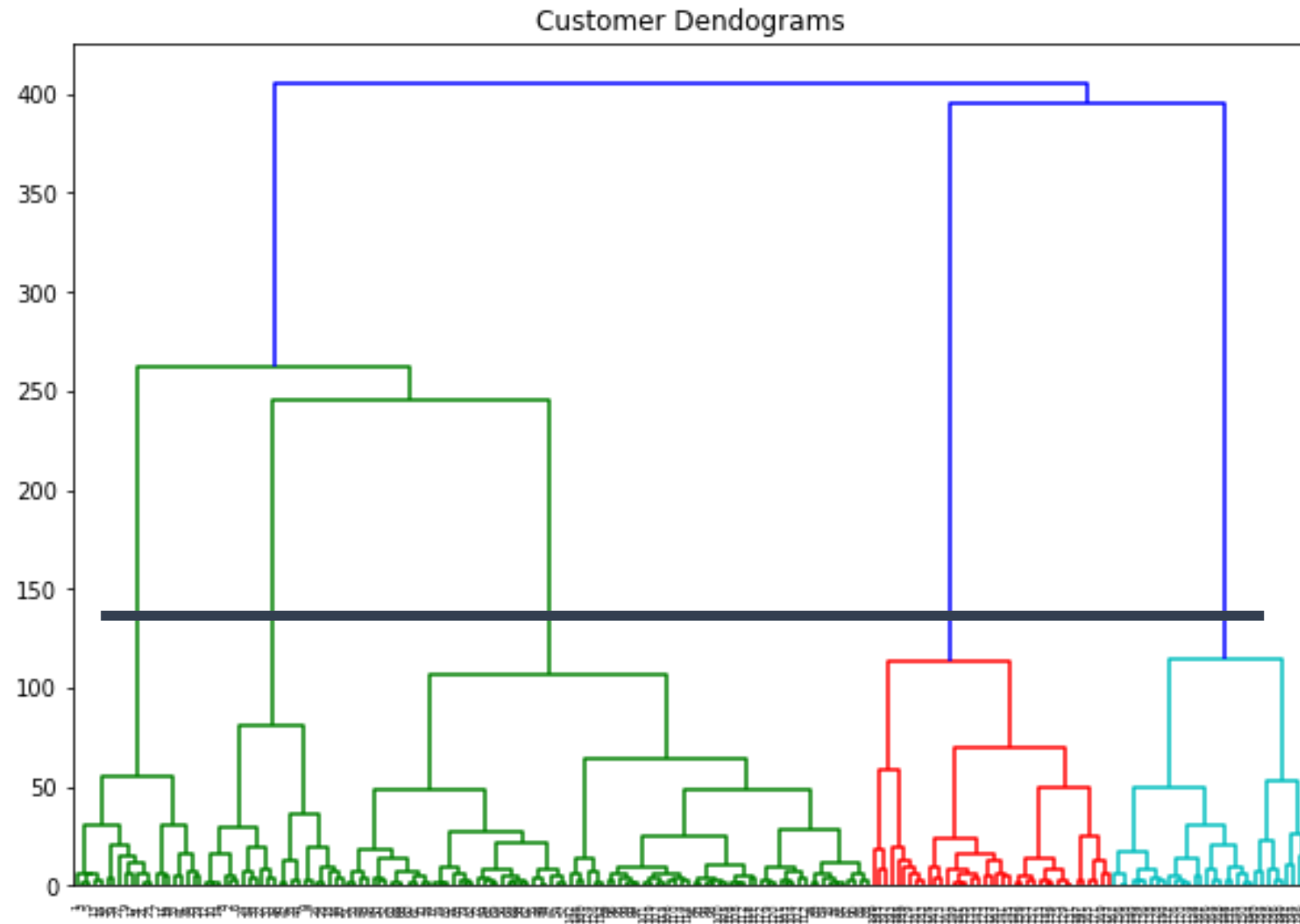
[0 0 0 0 0 0 1 1 1 1 1]

Example: Customer Shopping

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40



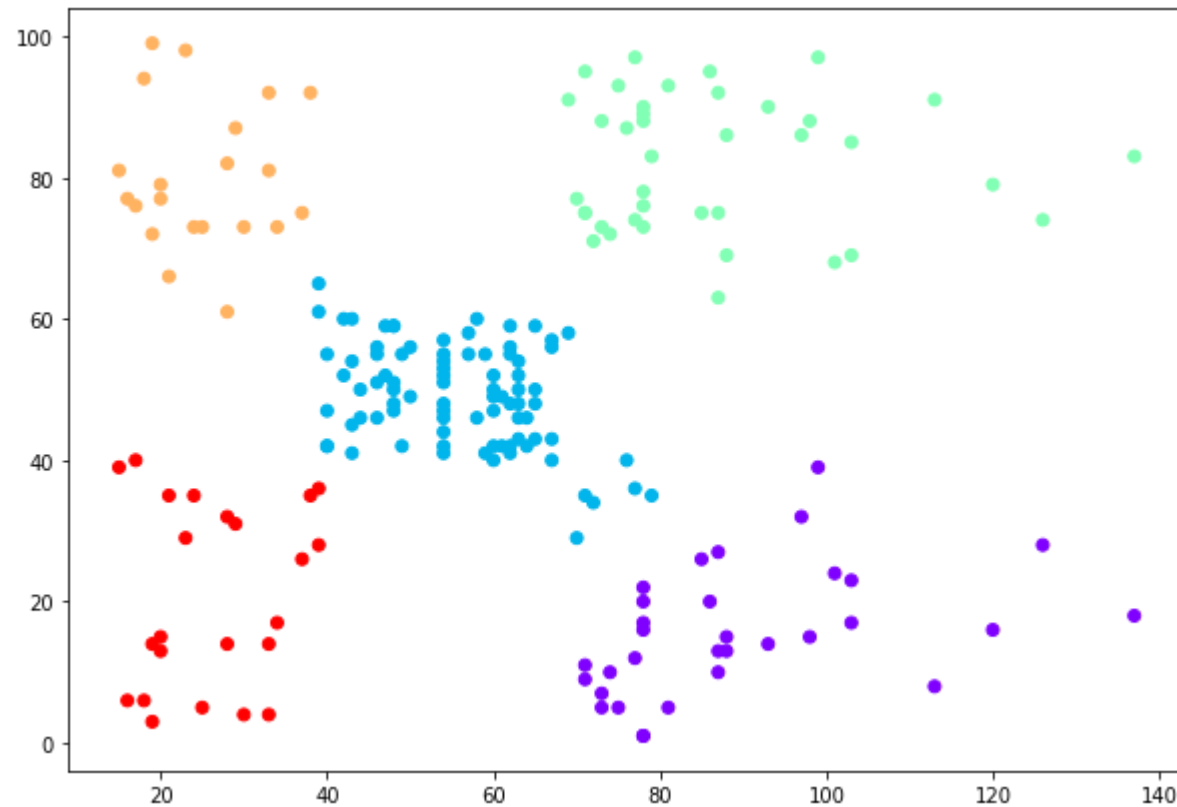
<https://stackabuse.com/hierarchical-clustering-with-python-and-scikit-learn>



Group the data into five clusters

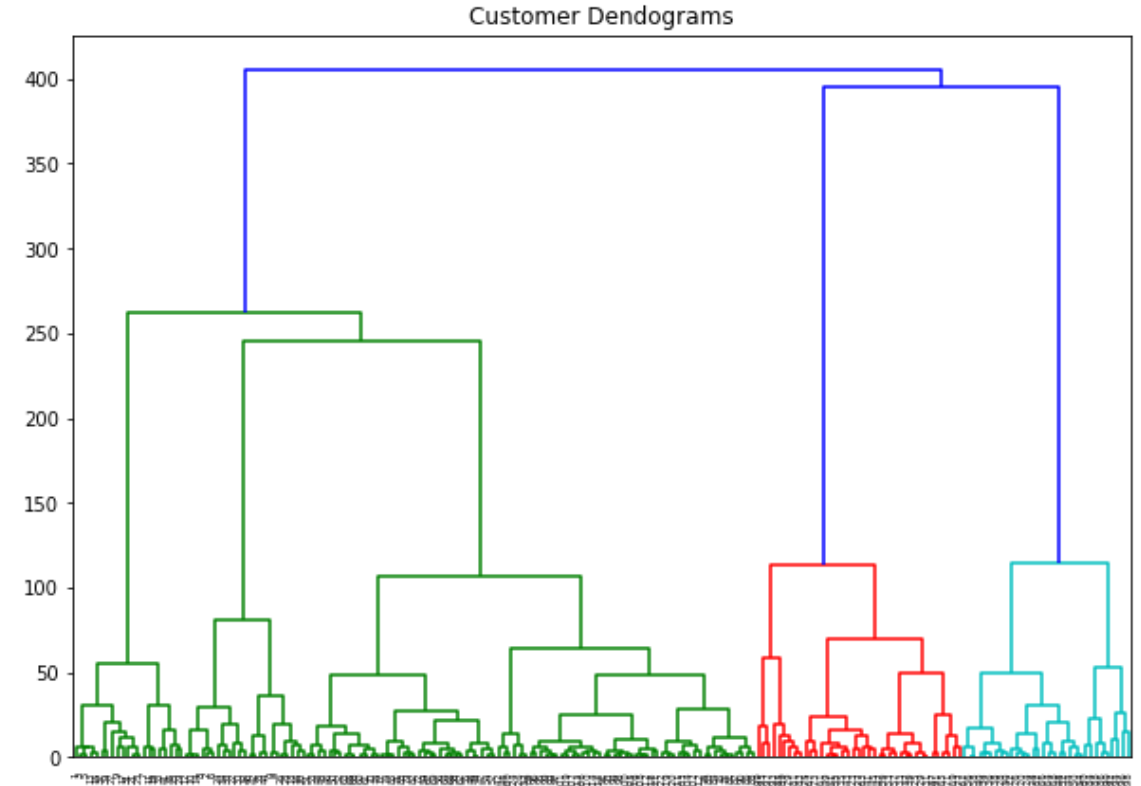
```
array([4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3,
4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3,
4, 3, 4, 3, 4, 1, 4, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 2, 1, 2, 1, 2, 0, 2, 0, 2, 1, 2, 0, 2, 0, 2,
0, 2, 0, 2, 1, 2, 0, 2, 1, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2,
0, 2, 1, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2,
0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2,
0, 2], dtype=int64)
```

Group the data into five clusters



Dendrograms

- The height on the vertical axis, indicates the (dis)similarity/distance between two clusters.
- The higher the height, the less similar the clusters are.
- This height is known as the *cophenetic distance*.



Note: conclusions about the proximity can be drawn only based on the height. The distance along the horizontal axis is not a criteria of similarity.

How to perform AHC?

- Data preparation
- Calculate **(dis)similarity** between every pair of objects.
- Compute **linkage** function to group objects clusters, based on the distance info generated in step 1. Use linkage function to link Objects/clusters in close proximity together.
- Determine where to cut the hierarchical tree into clusters.

Similarity Measures

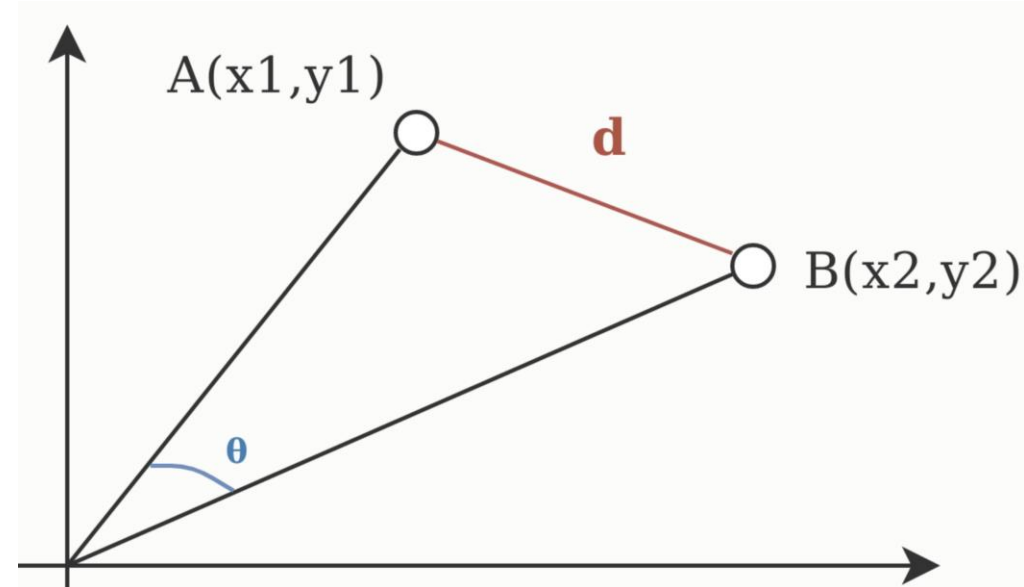
- Similarity measures are used to determine how much two objects are alike.
- **Different ways to calculate dis(similarity):**
 - 1) Cosine Similarity
 - 2) Manhattan distance
 - 3) Euclidean distance
 - 4) Minkowski distance: a generalization of the Euclidean and Manhattan distances.
 - 5) Jaccard similarity

Similarity Measures

- **Different ways to calculate similarity:**

- 1) Cosine Similarity
- 2) Manhattan distance
- 3) Euclidean distance
- 4) Minkowski distance

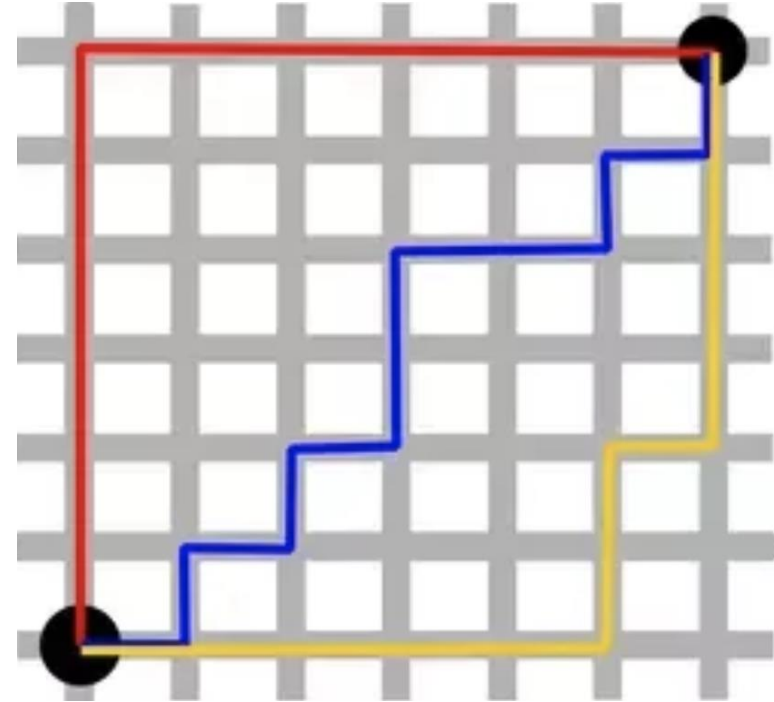
$$\text{Cos}\theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|} = \frac{\sum_1^n a_i b_i}{\sqrt{\sum_1^n a_i^2} \sqrt{\sum_1^n b_i^2}}$$



Similarity Measures

- **Different ways to calculate similarity:**

- 1) Cosine Similarity
- 2) **Manhattan distance**
- 3) Euclidean distance
- 4) Minkowski distance



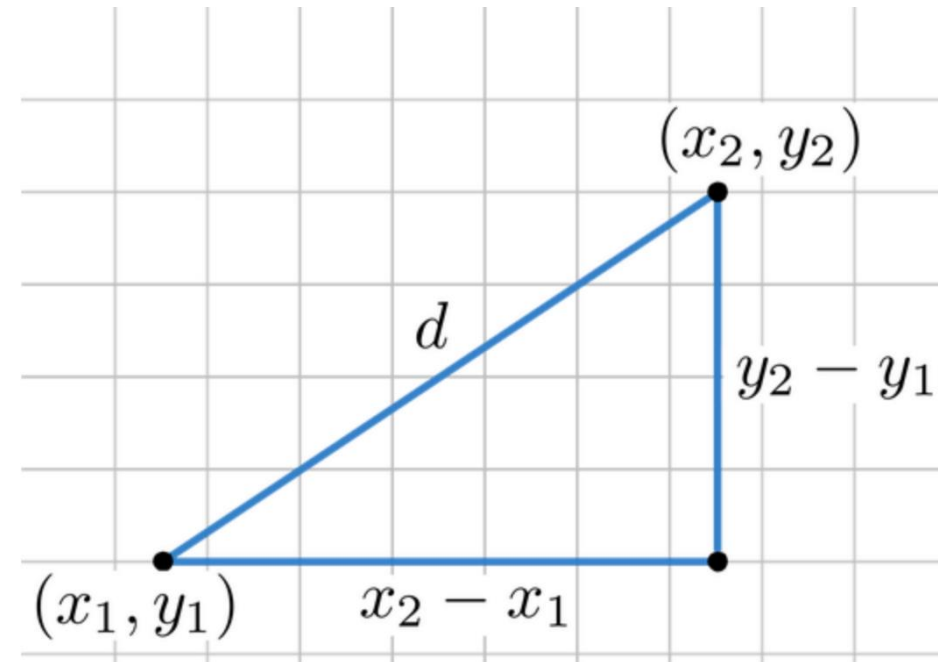
$$|x_1 - x_2| + |y_1 - y_2|$$

Similarity Measures

- **Different ways to calculate similarity:**

- 1) Cosine Similarity
- 2) Manhattan distance
- 3) Euclidean distance
- 4) Minkowski distance

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$



Similarity Measures

- **Different ways to calculate similarity:**
 - 1) Cosine Similarity
 - 2) Manhattan distance
 - 3) Euclidean distance
 - 4) Minkowski distance

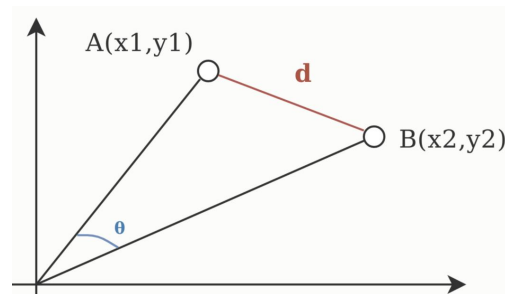
$$X = (x_1, x_2, \dots, x_n) \text{ and } Y = (y_1, y_2, \dots, y_n) \in \mathbb{R}^n$$

$$D(X, Y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

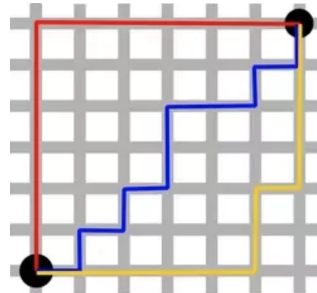
Similarity Measures

- **Different ways to calculate similarity:**

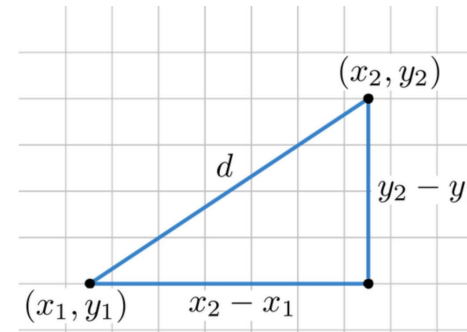
- 1) Cosine Similarity
- 2) Manhattan distance
- 3) Euclidean distance
- 4) Minkowski distance



$$\cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|} = \frac{\sum_1^n a_i b_i}{\sqrt{\sum_1^n a_i^2} \sqrt{\sum_1^n b_i^2}}$$



$$|x_1 - x_2| + |y_1 - y_2|$$



$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

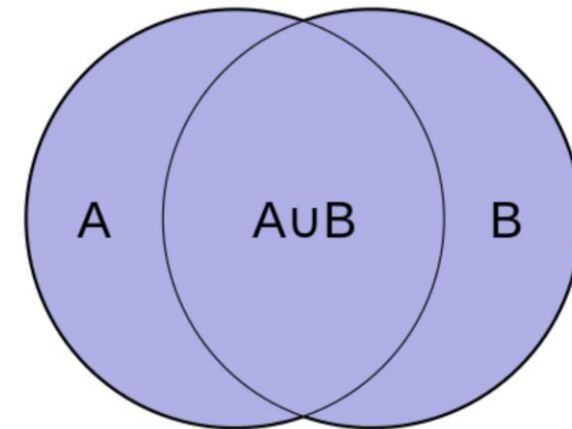
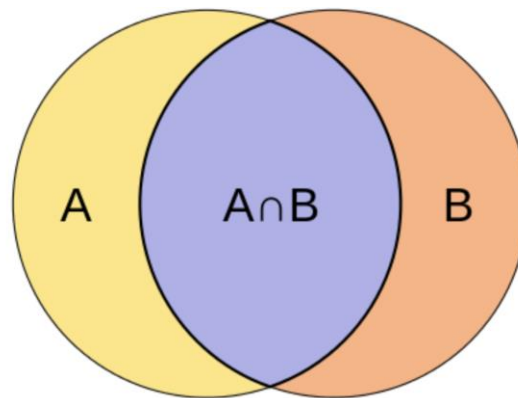
$$D(X, Y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

Similarity Measures

- **Different ways to calculate similarity:**

- 1) Cosine Similarity
- 2) Manhattan distance
- 3) Euclidean distance
- 4) Minkowski distance
- 5) Jaccard similarity

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$



Linkage Methods

- **Maximum or complete linkage:** The maximum value of all pairwise distances between data points in cluster 1 and in cluster 2. (produce compact clusters)
- **Minimum or single linkage:** The minimum value of all pairwise distances between the data points in cluster 1 and the data points in cluster 2 (Produce long, “loose” clusters).
- **Mean or average linkage:** The average distance between the elements in cluster 1 and cluster 2.
- **Centroid linkage:** The distance between the centroid for cluster 1 and the centroid for cluster 2.
- **Ward’s method:** It minimizes the total within-cluster variance. At each step the pair of clusters with minimum between-cluster distance are combined.

Example: A novel global urban typology framework for sustainable mobility futures

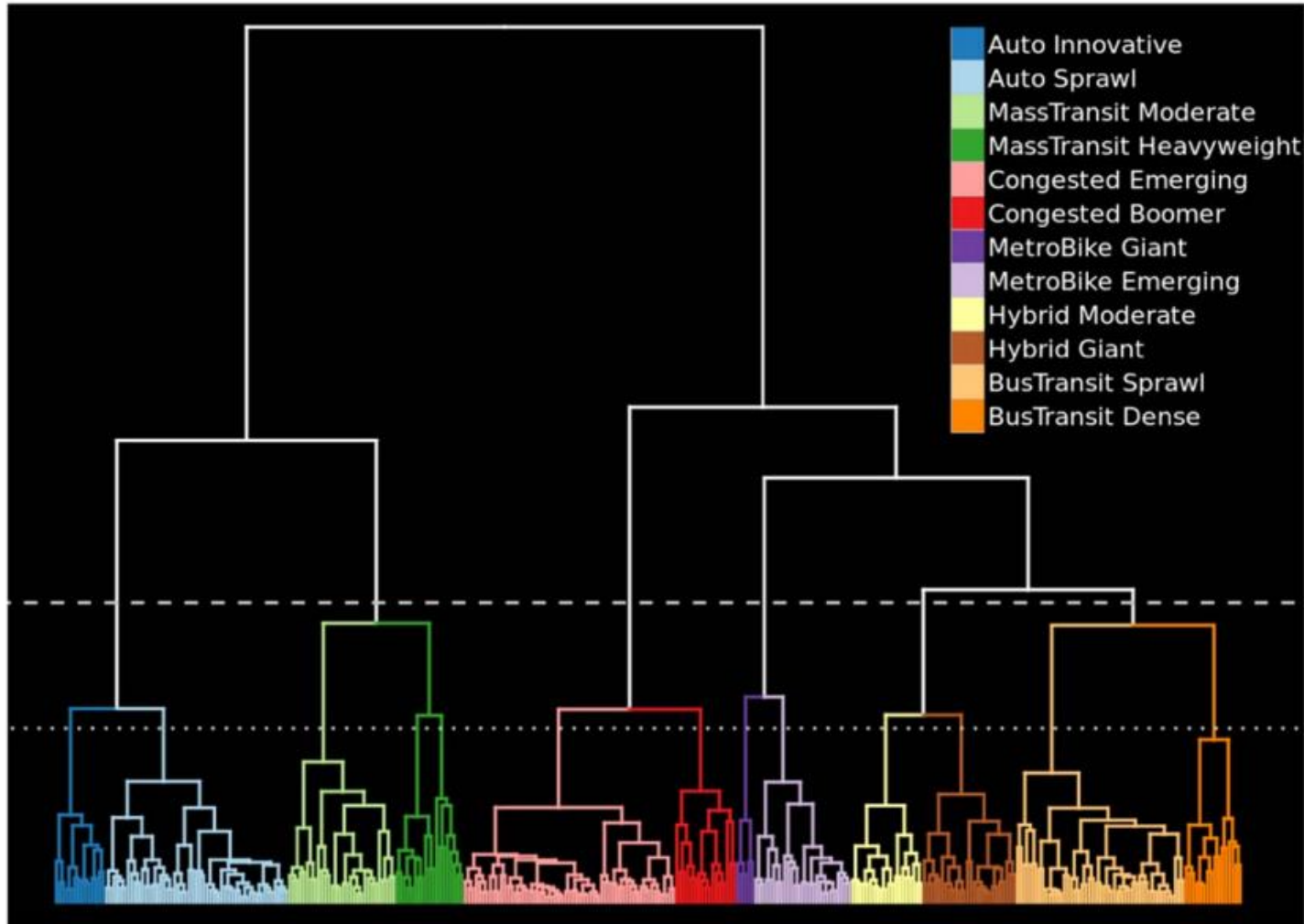
- Urban typologies for understanding dynamics of cities.
- **Dataset:** The paper presents a new typologization spanning 331 cities in 124 countries. The sample represents 40% of the global urban population and contains data from 2008.
- **Approach:** a factor analytic and AHC,
- **Results:** identify 9 urban factors and 12 typologies.

Example: A novel global urban typology framework for sustainable mobility futures

Summary of latent urban factors

Factor	Key indicators
Metro propensity	Urban rail/metro (demand, supply, age)
BRT propensity	Bus rapid transit(demand, supply, fares)
Bikeshare propensity	Bikeshare (demand, supply), low cost of living
Development	Wealth, cost of living indices, innovation
Population	Growth, population change
Congestion	Congestion (various metrics), public transit mode share, low car mode share
Sustainability	Bike mode share, street length, safety, efficiency, low congestion
Sprawl	Road deaths, high car mode share, low gas price, CO ₂ emissions, street length
Network density	High intersection density, high street density, low street length average, low circuitry

Example: A novel global urban typology framework for sustainable mobility futures



Example: A novel global urban typology framework for sustainable mobility futures

Summary of the urban typologies and their key cities

Typology	No. cities	Features; major locations	Key example cities
Auto Innovative	14	Auto-dependent, wealthy, higher transit mode share, metro & population density; U.S., Canada	Washington DC, Boston, Chicago, San Francisco, Toronto
Auto Sprawl	51	Auto-dependent, wealthy, sprawling, lowest transit mode share; U.S., Canada, Middle East	Baltimore, Tampa, Raleigh, Kuwait City
BusTransit Dense	16	Large population, high BRT, fairly congested; South America	Bogota, Rio de Janeiro, Jakarta, Sao Paulo, Tehran
BusTransit Sprawl	47	Lower population, sprawling, fair public transit ; Latin America, Central Asia/Middle East	Mecca, Shiraz, Santa Cruz, Tripoli, Caracas
Congested Boomer	17	Rapid growth, congestion, moderate car mode share; Indian Subcontinent, Africa	Bangalore, Chennai, Delhi, Lagos, Manila
Congested Emerging	59	High growth, lower population, developing; Africa, S. Asia	Kumasi, Phnom-Penh, Port-au-Prince, Lucknow
Hybrid Giant	26	Mix of mode choices, dense networks, high population density; S./E. Europe, E. Asia	Busan, Lisbon, Sapporo, Santiago, Warsaw
Hybrid Moderate	20	Mix of mode choices, lower population; Central America, Middle East	Havana, Johannesburg, Montevideo, Panama City
MetroBike Emerging	27	Metro & bikeshare dominant, highway development, fairly wealthy; China	Ningbo, Zhengzhou, Shenyang, Harbin
MetroBike Giant	5	Metro & bikeshare dominant, large population; wealthy; China	Shenzhen, Guangzhou, Chongqing, Beijing
MassTransit Heavyweight	19	High mass transit usage and metro availability, high bike-share; fairly high CO ₂ emissions; Europe, S.E. Asia	Singapore, Madrid, Seoul, Berlin, London
MassTransit Moderate	30	Equitable, high bikeshare, moderate metro and BRT, low congestion; W. Europe, Israel	Antwerp, Tel Aviv, Turin, Liverpool